EXAMEN

Semestre : 1 ☐ 2 ■

Session : Principale ☐ Rattrapage ■

Unité d'enseignement : Equipe complexité
Module (s) : Complexité appliquée à la RO
Classe(s) : 4ème année

Nombre d'exercices : 3 Nombre de pages : 3

Date : 21/06/2023 Heure 15h Durée :1h30

Exercice 1 : (6points)

Déterminer la complexité de chaque séquence ci-dessous **en justifiant votre réponse.**

Seq1 : Pour i=1 à N faire S=0 M=N-i Pour j=1 à M faire S=S+j Fin pour Fin pour	**Seq2 :** S=0 Pour i=1 à N faire Si (i mod 2=0) N=N+1 Fin Si S=S+i Fin pour	**Seq3 :** i=1 S=0 Tant que (i<N) faire S=S+i N=N-1 i=i+1 Fin Tant que
Seq4 : S=0 Pour j=1 à N faire i=1 Tant que (i<N) faire i=i*2 S=S+i Fin Tant que Fin pour	**Seq5 :** S=0 Pour i=1 à N faire Pour j=i à N faire S=S+i Fin pour Fin pour	**Seq6 :** S=0 i=1 Si (N mod 3==0) Pour i=1 à N faire S=S+i Fin pour Sinon Tant que (i<N) faire i=i /3 S=S+i Fin Tant que Fin Si

Exercice 2

Soit le code ci-dessous écrit en langage C où la fonction **Recherche** permet de vérifier en premier lieu si le tableau donné en paramètre est trié à l'aide de la fonction **Est_trie.** S'il est trié, on effectue une recherche d'une valeur **x** dans le tableau en utilisant la méthode dichotomique (**Recherche_dichotomique),** sinon on effectue une recherche en utilisant la méthode séquentielle (**Recherche_sequentielle).**

1. Pour **n=4**, donner une proposition des valeurs des paramètres **tab** et **x** afin de maximiser le temps d'exécution du programme et donner d'autres valeurs qui le minimise.

2. Calculer la complexité du programme dans le pire et au meilleur des cas à O près (en terme de comparaison)

3. Déduire une optimisation du programme.

```
Void Recherche( int tab[], int n, int x)
{
int pos ;
if (Est_trie(tab,n)==1)
    pos=Recherche_dichotomique(tab,n,x) ;
else
    pos=Recherche_sequentielle(tab,n,x) ;
if(pos==-1)
    printf(''la valeur %d non trouvable dans le tableau'', x) ;
else
    printf(''la valeur %d existe dans la position %d  du tableau'',x,pos) ;
}
```

```
int Est_trie (int tab[],int n)
{  int i=0, trie=1 ;
   while (trie==1 && i<n-2)
    {   if (tab[i] <= tab[i+1])
           i++ ;
        else
           trie=0 ;
    }
     return trie ;
}
```

Exercice 3 :

On se propose de calculer le Produit Matriciel des deux matrices carrées **A** et **B** de taille ***n***. Soit ***C*** la matrice carrée de taille ***n*** résultantes. Pour ce faire, ils existent plusieurs algorithmes.

1. Un premier algorithme itératif appelé **AlgoPM_naïf (ci-dessous)** Calculer la complexité de **AlgoPM_naïf**

```
AlgoPM_naïf (A, B,C)
   For i = 1 to n do
       For j=1 to n do
           C(i,j)=0
           For k=1 to n do
                C(i,j)= C(i,j) +A(i,k)*B(k,j)
           End For
        End For
    End For
End_ AlgoPM_naïf
```

2. Un deuxième algori viser pour Régner » appelé **AlgoPM_Rec**. Cet algorithme décompose les matrices ***A***, ***B*** et ***C*** en sous-matrices de taille *n*/2**n*/2 comme suit :

$$A = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}; B = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}; C = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}$$

L'équation $C = A*B$ peut alors se récrire :

$$\begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix} * \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix};$$

avec $c_{11}=a_{11}*b_{11}+a_{12}*b_{21}$
$c_{12}=a_{11}*b_{12}+a_{12}*b_{22}$
$c_{21}=a_{21}*b_{11}+a_{22}*b_{21}$
$c_{22}=a_{21}*b_{12}+a_{22}*b_{22}$

Calculer le nombre d'opérations de multiplications de matrices carrées de taille n/2 et le nombre d'addition de telles matrices. Donner alors l'équation de récurrence de la complexité de **AlgoPM_Rec** puis la calculer.

3. Un troisième algorithme récursif semblable à **AlgoPM_Rec** appelé **AlgoPM_Strassen.** Cet algorithme applique la méthode de Strassen utilisant **7** nouvelles matrices $\mathbf{M_i}$ qui servent à exprimer les $\mathbf{c_{ij}}$ avec uniquement 7 multiplications au lieu de 8.

$M_1=(a_{11}+a_{22})*(b_{11}+b_{22})$
$M_2=(a_{21}+a_{22})*b_{11}$
$M_3=a_{11}*(b_{12}-b_{22})$
$M_4=a_{22}*(b_{21}-b_{11})$
$M_5=(a_{11}+a_{12})*b_{22}$
$M_6=(a_{21}-a_{11})*(b_{11}+b_{12})$
$M_7=(a_{12}-a_{22})(b_{21}+b_{22})$

Les c_{ij} sont alors exprimées comme suit :
$c_{11}=M_1+M_5-M_5+M_7$
$c_{12}=M_3+M_5$
$c_{21}=M_2+M_4$
$c_{22}=M_1-M_2+M_3+M_6$

Donner la formule de récurrence de la complexité de **AlgoPM_DeStrassen** puis la calculer.

